Causal statistics for treatment models

8. An introduction to machine learning for causal effect estimation

Alexandre Grellet

Ecole Normale Supérieure–PSL, L3 Economics and Cogmaster, 2025

Table of contents

Today

We will explore machine learning techniques. At the end of this lecture, you should:

- Know the motivation for using machine learning techniques for causal inference;
- Understand the differences between supervised and unsupervised learning;
- Understand the basics behind regression trees and random forests;
- Know the advantages and limits of causal forests for the estimation of conditional average treatment effects.

Introduction

What is machine learning?

- Machine learning (ML) as a **subdomain of AI** that aims to develop algorithms and statistical models that enable computers to perform specific tasks without explicit programming.
- Can be supervised (prediction, classification) or unsupervised (clustering).

Some vocabulary you might encounter in the literature

- **Features**: Variables or attributes of data.
- **Labels**: The target variable we aim to predict.
- **Models**: Algorithms that learn from data.

Some definitions

Unsupervised learning

The goal of unsupervised learning is to **identify underlying structures or clusters in the data**, which are not already known to the researcher.

Supervised learning

The goal of supervised learning is to train an algorithm to make it able to **predict value for an outcome given features**, so that the algorithm can make predictions on new, unseen data.

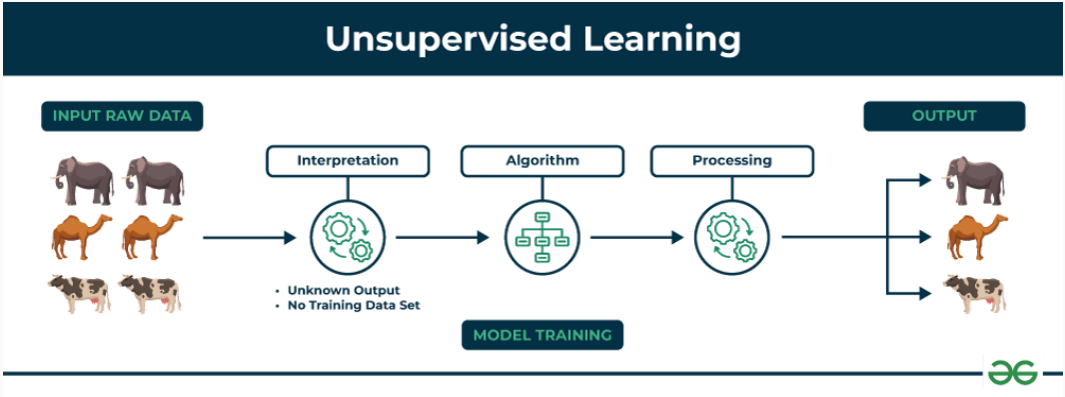
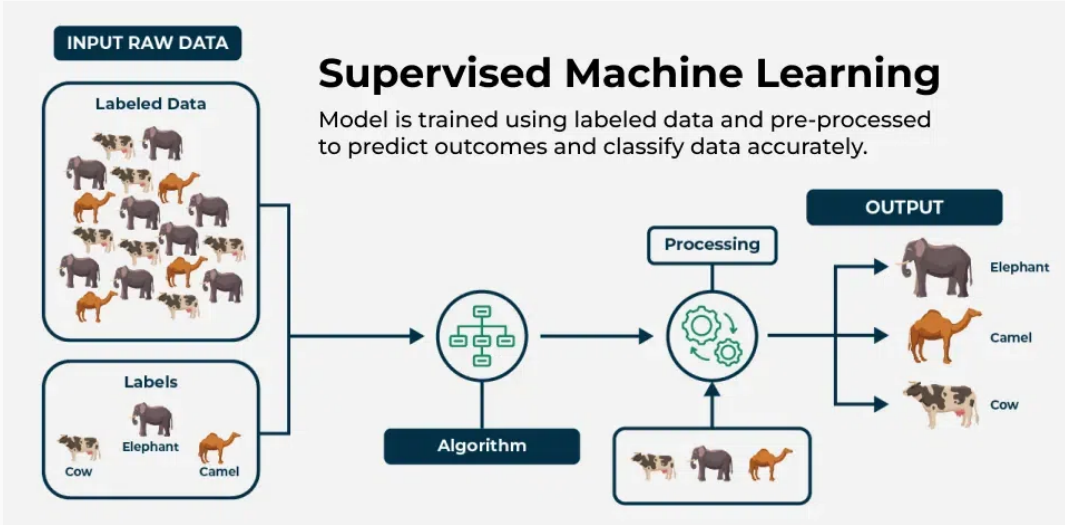


Figure 1: Figure from GeeksforGeeks

Using unsupervised learning

- Descriptive statistics;
- Finding stylized facts and identifying patterns.

We will NOT discuss unsupervised learning today. For a state-of-the-art review on the use of unsupervised learning in economics, you can read Dell (2025).


INPUT RAW DATA
Labeled Data
Labels
Cow
Elephant
Camel
Supervised Machine Learning
Model is trained using labeled data and pre-processed to predict outcomes and classify data accurately.
Algorithm
Processing
OUTPUT
Elephant
Camel
Cow

Main use of supervised learning

- Classification;
- Estimate the relation between outcome and covariates (most of the time not causal – very good algos for prediction);
- Estimate an ATE (e.g. regression);
- Estimate **conditional average treatment effects** (CATEs) = estimate heterogeneous effects $\rightarrow$ Algos to find subsets of the population with different causal effect for a same policy.

Today, we will discuss the last point. We will not discuss thoroughly early models relying on **sparsity**.

A brief word about sparsity

Sparsity

Sparsity refers to having a large number of zeros (or near-zeros) in data vectors or model parameters.

What this means: only a few elements (or features) contribute significantly, while the rest are negligible.

Example: in the model $Y = \alpha + \sum_{j=1}^{J} \beta_j X_j$, this would imply that only a few of the X_js are non-zero.

A well-known algorithm: LASSO Regression

Objective Function

$$\min_{\beta} \left\{ \frac{1}{N} (Y - X\beta)^2 + \lambda|\beta| \right\}$$

- The absolute value penalty induces sparsity in the parameter vector β.
- LASSO is widely used nowadays. Its asymptotic distribution and properties are quite complicated. We will not study it today $\rightarrow$ you will have classes about it if you take M2 courses in econometrics.

Motivation for using machine learning

- Large datasets (lot of covariates + lot of observations).

Explanations

Motivation for using machine learning

- Large datasets (lot of covariates + lot of observations).
  Explanations
- Unknown functional form of the relation between a variable Y and predictors X.

Motivation for using machine learning

- Large datasets (lot of covariates + lot of observations).

  Explanations

- Unknown functional form of the relation between a variable Y and predictors X.
- Heterogeneity analysis without assuming a priori dimensions of heterogeneity.

Focus of this class = supervised learning

We will focus on the case when we have data coming from an unknown data generating process $f(X)$ (= we don't know the true distribution of the outcome respective to the treatment and the covariates), and we want to estimate the correct functional form. In general, this problem is to estimate:

$$Y = \widehat{f}(X)$$

Focus of this class = supervised learning

We will focus on the case when we have data coming from an unknown data generating process $f(X)$ (= we don't know the true distribution of the outcome respective to the treatment and the covariates), and we want to estimate the correct functional form. In general, this problem is to estimate:

$$Y = \widehat{f}(X)$$

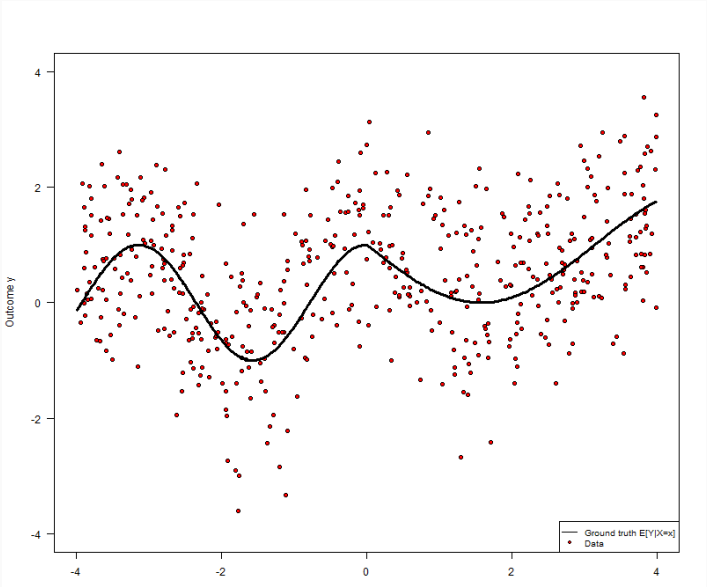
When we try to estimate the effect of a treatment T, this is:

$$Y = \widehat{f}(T, X)$$

Contrary to usual techniques, the functional form f is not assumed to take a certain form. ML algorithm will aim to estimate it.

In OLS regression, what is the functional form of f?

ML is particularly good for non-linear relationships



Focus of this class = supervised learning

$$Y = \widehat{f}(T, X)$$

In our case, the function we want to estimate is the **Conditional average Treatment Effect (CATE)**:

$$\tau(X) = \mathbb{E}\left(Y(1) - Y(0) \mid X = x\right)$$

The set-up

To run machine learning techniques, we need:

- A dataset $(Y_i, X_i)_{i=1:N}$.
- A prediction algorithm.
- A loss function $l(\widehat{Y}, Y)$ to assess the difference between the predicted value and actual outcome.

With OLS, what is the loss function?

The loss function

With OLS, we want to minimize the sum of square residuals:

$$l(\widehat{Y}, Y) = \mathbb{E}\left(\left(\widehat{Y} - Y\right)^2\right)$$

The loss function

With OLS, we want to minimize the sum of square residuals:

$$l(\widehat{Y}, Y) = \mathbb{E}\left(\left(\widehat{Y} - Y\right)^2\right)$$

We obtain predicted values $X\widehat{\beta}$. Assuming the true model is indeed the linear one, we have: $Y = X\beta + \epsilon$. The estimates are obtained with the dataset at hand $(Y_i, X_i)_{i=1:N}$. But ML is overall interested in prediction.

The loss function

With OLS, we want to minimize the sum of square residuals:

$$l(\widehat{Y}, Y) = \mathbb{E}\left(\left(\widehat{Y} - Y\right)^2\right)$$

We obtain predicted values $X\widehat{\beta}$. Assuming the true model is indeed the linear one, we have: $Y = X\beta + \epsilon$. The estimates are obtained with the dataset at hand $(Y_i, X_i)_{i=1:N}$. But ML is overall interested in prediction.

Say we have an out-of-sample observation, the expected loss at this new point with OLS is:

$$\begin{aligned}\mathbb{E}\left(\left(\widehat{Y} - Y\right)^2\right) &= \mathbb{E}\left(\left(X\widehat{\beta} - X\beta + \epsilon\right)^2\right) \\ &= \underbrace{\left(X(\beta - \mathbb{E}(\widehat{\beta})\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\left(\left(\widehat{Y} - \mathbb{E}\left(\widehat{Y}\right)\right)^2\right)}_{\text{Variance estimation}} + \underbrace{\mathbb{E}(\epsilon^2)}_{\text{Noise}}\end{aligned}$$

Proof

The loss function

$$\mathbb{E}\left(\left(\widehat{Y}-Y\right)^2\right)=\underbrace{\left(X(\beta-\mathbb{E}(\widehat{\beta})\right)^2}_{\text{Bias}^2}+\underbrace{\mathbb{E}\left(\left(\widehat{Y}-\mathbb{E}\left(\widehat{Y}\right)\right)^2\right)}_{\text{Variance estimation}}+\underbrace{\mathbb{E}(\epsilon^2)}_{\text{Noise}}$$

The loss function

$$\mathbb{E}\left(\left(\widehat{Y}-Y\right)^2\right)=\underbrace{\left(X(\beta-\mathbb{E}(\widehat{\beta})\right)^2}_{\text{Bias}^2}+\underbrace{\mathbb{E}\left(\left(\widehat{Y}-\mathbb{E}\left(\widehat{Y}\right)\right)^2\right)}_{\text{Variance estimation}}+\underbrace{\mathbb{E}(\epsilon^2)}_{\text{Noise}}$$

The decomposition of the bias highlights:

- That one should reduce bias to reduce prediction error;
- but also reduce variance of estimated coefficients across samples.

ML focuses a lot on the second possibility, and computer scientist gave it a name: when $\mathbb{E}\left(\left(\widehat{Y}-\mathbb{E}\left(\widehat{Y}\right)\right)^2\right)$ is high, we say the model is overfitting the data.

Why is overfit such a concern with ML?

The concern for overfitting

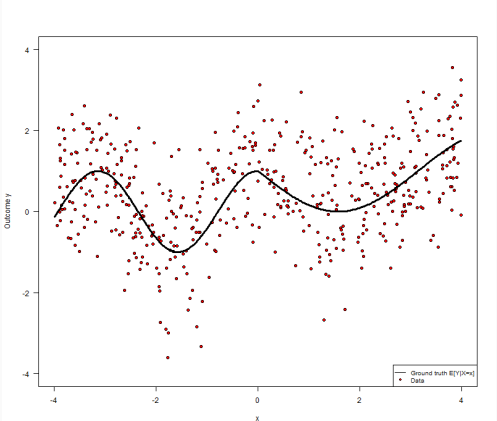
A lot of ML techniques are highly non-parametric $\Leftrightarrow$ the estimated functional form is very flexible.

But if $\widehat{f}(\cdot)$ can move too freely, the risk is that it sticks too well to the data at hand. Hence, we need to limit expressiveness with ML: this is called **regularization**.

The choice of a specific regularizer is often called **tuning**.

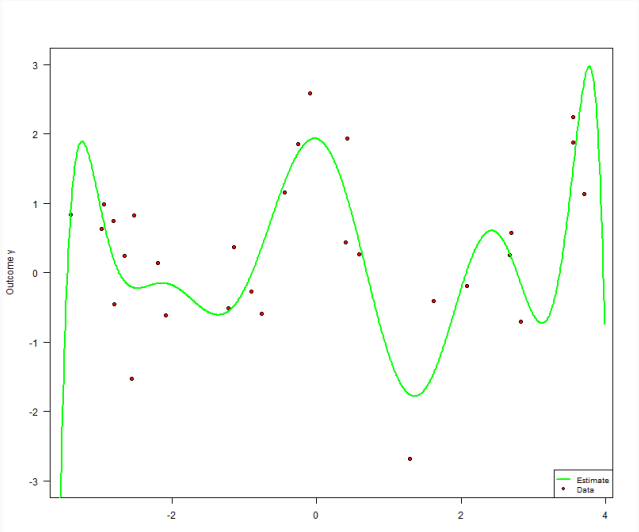
Illustrating overfits (borrowing from Golub Capital Social Impact Lab)

Say we want to estimate the relationship on the previous graph with a polynomial (but we are unsure about the best degree for the polynomial). Overfit is when your estimation is too close to the sample at hand, so close actually that it departs from the superpopulation the data is coming from.



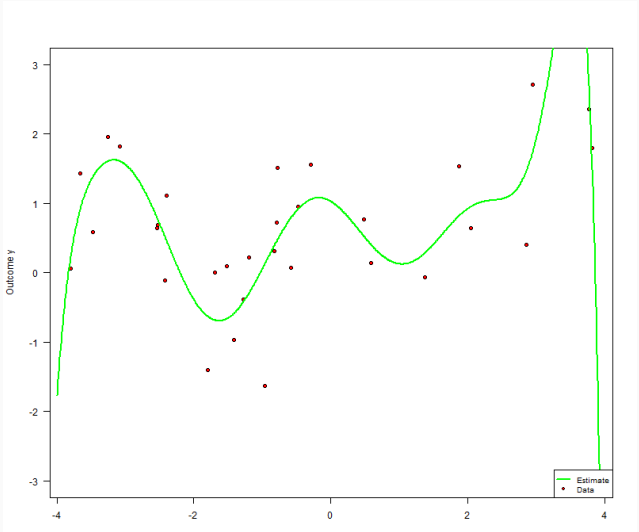
Illustrating overfits (borrowing from Golub Capital Social Impact Lab)

30 data points. 10th degree polynomial.



Illustrating overfits (borrowing from Golub Capital Social Impact Lab)

30 other data points. 10th degree polynomial.



Illustrating overfits (borrowing from Golub Capital Social Impact Lab)

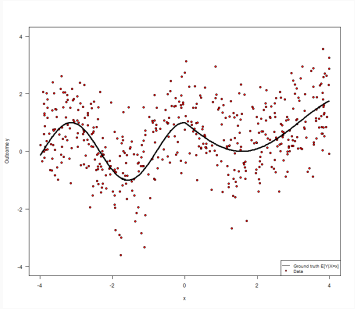


Figure 3: Ground truth

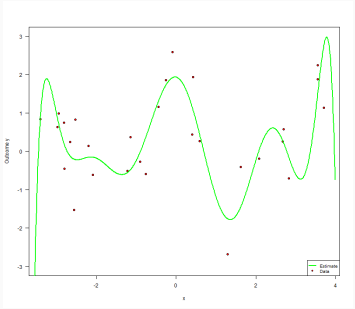


Figure 4: First estimation

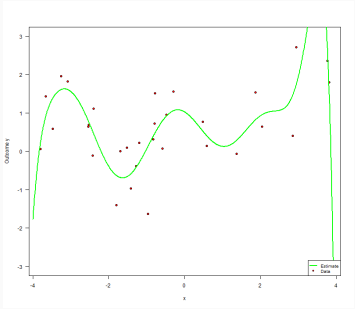
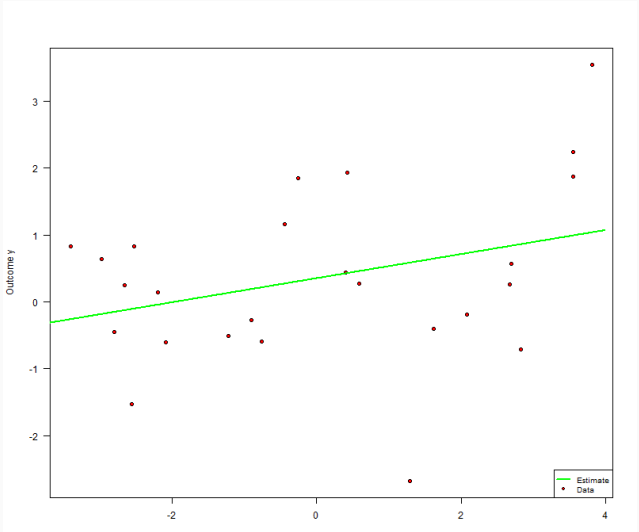


Figure 5: Second estimation

Illustrating underfits (borrowing from Golub Capital Social Impact Lab)

A simple linear regression.



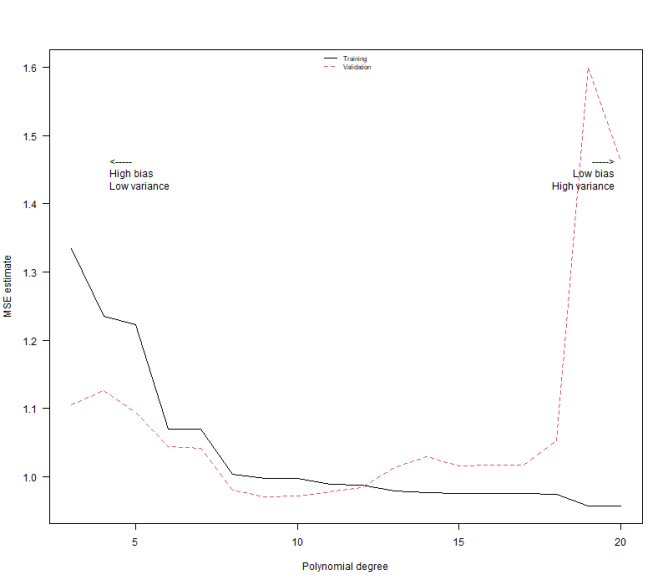
Any idea about ways to avoid at the same time under- and over-fitting?

Training and test datasets

Idea = Take your sample. Divide it in 2 parts. On the first part (called the **training dataset**), estimate your model. On the second part (called the **test dataset**), predict the values of the outcome based on the model you just estimated.

Find a model with a **good balance of bias** (low bias = loss function is low in the training dataset) **and variance** (low variance = loss function is low in the test dataset).

Finding the best fit



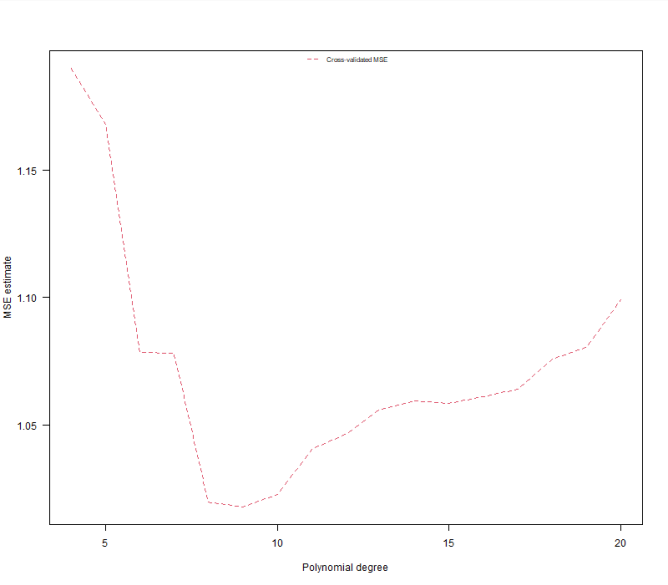
Cross-validation

Often, you want to test multiple models (= find which one has the best bias-variance tradeoff). In these cases, people tend to use what is called **K-fold cross validation**.

K-fold cross validation =

- Divide the data into K subsets (folds).
- Fit K times your model, each time using the data from $K-1$ folds for estimation and then evaluate the fitted model on the remaining fold (called the **validation set**).
- Average the MSE ("cross-validated MSE").
- Best model is the one that reduces the most the cross-validated MSE.

Cross-validation



Common prediction algorithms

- Regularized linear models (LASSO, ridge regression, DDML);
- Decision trees;
- Forest.

Today, we will only discuss decision trees and forests $\rightarrow$ they are suited for inference (provided we use the good loss function and algorithm).

Decision Trees and Regression Trees

What are Decision Trees?

- A decision tree is a **series of decisions in a tree-like structure**.
- Useful for both **classification and regression**.
- Built through **recursive splits on features**, aiming to reduce prediction error.

A classification tree

- 2 values (labels) for the categorical outcome: the student is good (A), the student is not good (B);
- 2 features: $G \equiv$ Grade, $S \equiv \mathbb{1}$ (Student is sleeping during the course).
- Left of the branch means false, right means true.

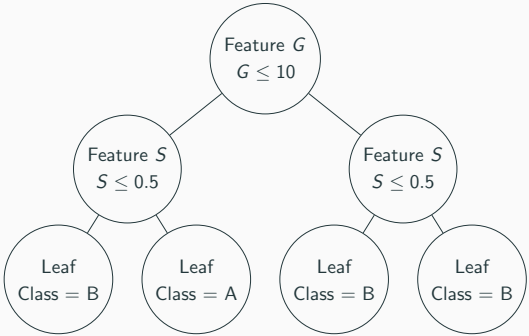


Figure 6: Example Decision Tree: classifying good students

Main idea of a tree

Idea: you want to predict (no causation at this point) a binary outcome variable Y, with features X.

Algorithm idea:

- For all variables, test thresholds to split the dataset.
- Select the variable and threshold such that the average predicted values differ as much as possible on each side of the cutoff.
- Split the tree according to the chosen variable and threshold.
- Repeat this procedure with the branches you just created. Stop when no more partition can be made (or when you seem to start overfitting).

Back to the example

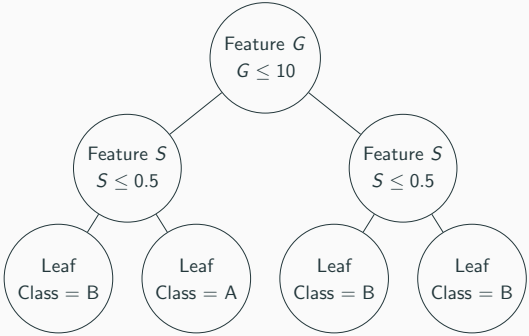
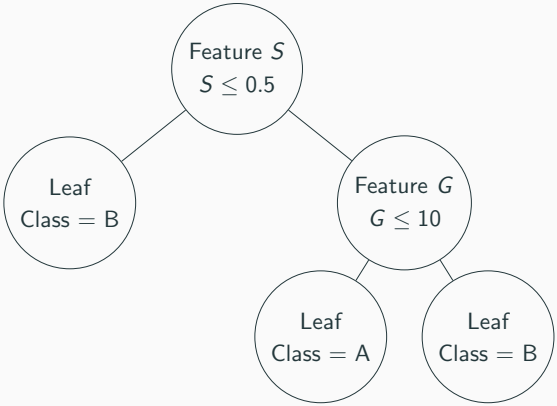


Figure 7: Example Decision Tree: classifying good students

Did we choose the best split? (Only give intuitions for the time being)

A classification tree

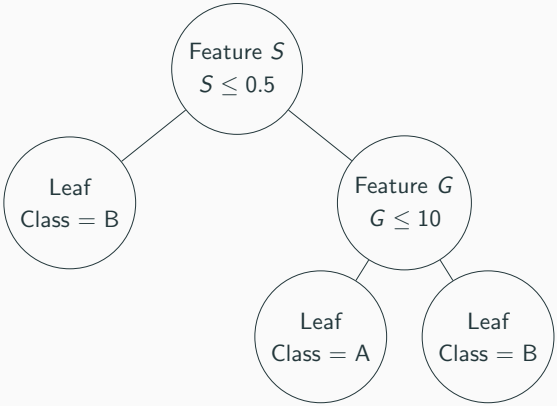
Actually, it is very likely that students who sleep in the course do not perform well, and that the following representation is more accurate:



In practice, how should we decide on the splits?

A classification tree

Actually, it is very likely that students who sleep in the course do not perform well, and that the following representation is more accurate:



In practice, how should we decide on the splits?Loss function + cross-validation!

Mathematics of Regression Trees: quantitative outcome

Disclaimer: we stop using classification for now, because loss function is more complicated, but idea is similar.

Mathematics of Regression Trees: quantitative outcome

Disclaimer: we stop using classification for now, because loss function is more complicated, but idea is similar.

- We use Mean Squared Error (MSE) to select splits:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

  where $\hat{y}_i$ is the average value of the outcome in the node where i is assigned.

- Recursive binary splitting: at each step you need to split in the tree, test thresholds for all variables, select the split that minimizes the within-node variance of the outcome. $\Rightarrow$ Find splits such that (i) lot of variations in Y across nodes, (ii) low variation within node.

Performing these successive splits is called **growing the tree**.

Exercising

Let's practice. Download the dataset called "ML_reg_tree.parquet" on Moodle and load it in R. Also install package rpart.

```
install.packages("rpart")
library(arrow)
library(rpart)
df <- read_parquet("ML_reg_tree.parquet")
```

Exercising

In R, use the function:

```
tree <- rpart(formula, data=df, cp=0, method="anova")
```

Formula must be of the same form as in a traditional regression in R: outcome $\sim$ sum of covariates. Your aim = predict the LOGVALUE of a house using features in the dataset (*Hint: you are allowed to use all features*).

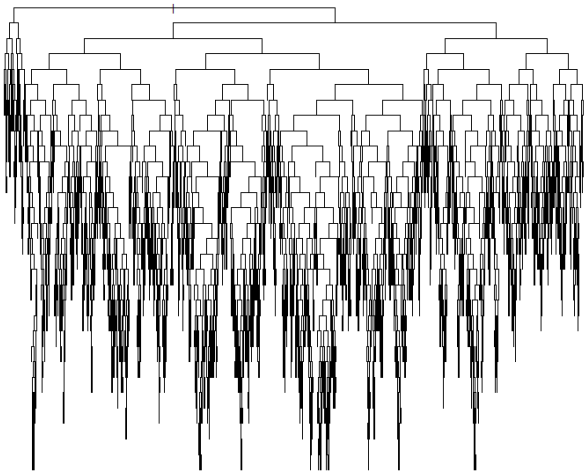
Solution

```r
# get all covariates
covariates <- colnames(df)

# remove outcome
covariates <- covariates[- which(covariates == "LOGVALUE")]

# Build tree
fmla <- formula(paste("LOGVALUE ~", paste(covariates, collapse=" + "))
    )
tree <- rpart(formula = fmla, data=df, cp=0, method="anova")
```

What a nice tree... or is it too large?



Avoiding overfitting

To avoid overfitting, we typically **prune the tree** (we give it a penalty for having too much branches). Let's call $\mathcal{T}(X)$ the value predicted by the tree $\mathcal{T}$ for a unit with features X.

We modify the tree such that the loss function becomes:

$$\widehat{\mathcal{T}} = \arg\min_{\mathcal{T}} \sum_{i=1}^{N} \left(\mathcal{T}(X_i) - Y_i\right)^2 + c_p|\mathcal{T}| \tag{1}$$

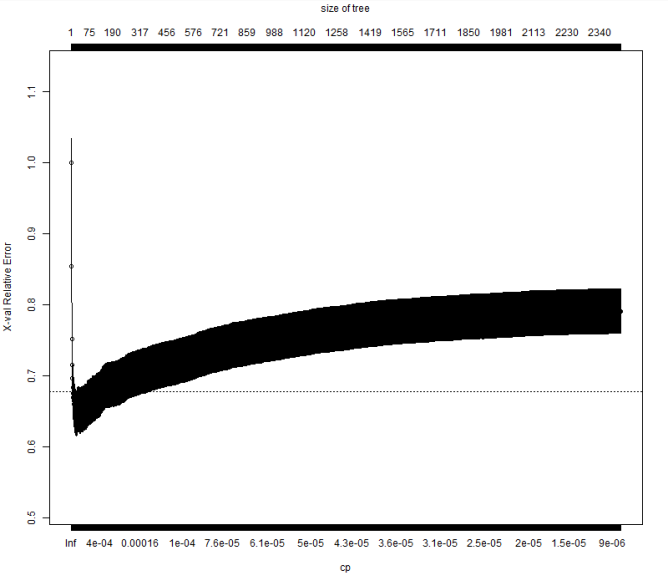
with $|\mathcal{T}|$ the number of leafs in the tree, and c_p a penalty for the number of leafs. The higher c_p, the lower the number of leafs.

Cross-validation and adjusted mean-squared-error

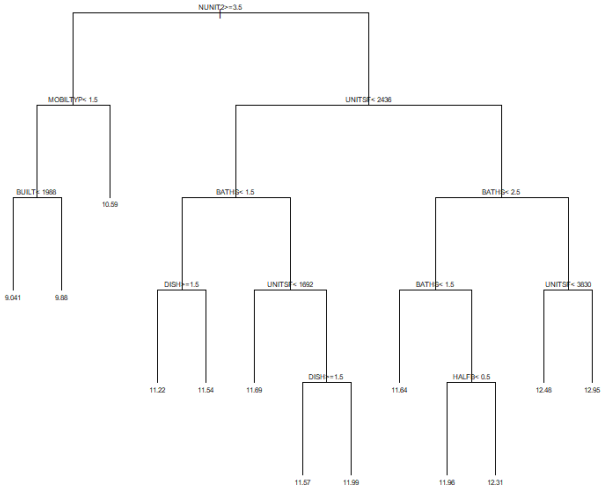
In R, use:

```
1        plotcp(tree)
```

Cross-validation and adjusted mean-squared-error



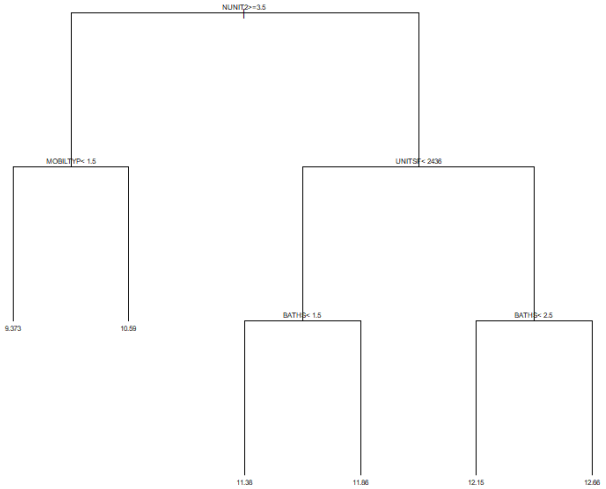
Pruned tree


NUNIT2>=3.5
MOBILTYP< 1.5
UNITSI< 2436
BUILT< 1988
10.59
BATHR< 1.5
BATHR< 2.5
9.041
9.68
DISH>=1.5
UNITSI< 1692
BATHR< 1.5
UNITSI< 3830
11.22
11.54
11.89
DISH>=1.5
11.64
HALFB< 0.5
12.48
12.95
11.57
11.99
11.96
12.31

Rule of thumb

Sometimes, people are still afraid that the complexity that minimizes the cross-validation (i) still overfits, or (ii) makes the tree uninterpretable. It is common that the following rule of thumb is used: take the complexity estimate that is the largest within the 1 standard error radius of the estimated minimizing complexity.

Pruned tree following rule of thumb



Questions?

Conditional average treatment effects

Estimating Conditional Average Treatment Effects

- **Conditional average Treatment Effect (CATE)**:

$$\tau(X) = \mathbb{E}[Y \mid X \in I, T = 1] - \mathbb{E}[Y \mid X \in I, T = 0]$$

  for I some leaf of the tree.
- Where T is treatment ($1 =$ treated, $0 =$ not treated) and X represents features / covariates.

It is the average treatment effect conditional on having characteristics X and thus ending up in leaf I.

Regression trees to estimate CATEs

Main difference: change the loss function. Ideally, what do you think is the loss function we would like to minimize?

Regression trees to estimate CATEs

Main difference: change the loss function. Ideally, what do you think is the loss function we would like to minimize?

We would like to maximize the difference in the treatment effect across nodes $\rightarrow$ suggests the following loss function:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (\tau_i - \widehat{\tau}(X_i))^2$$

Regression trees to estimate CATEs

Main difference: change the loss function. Ideally, what do you think is the loss function we would like to minimize?

We would like to maximize the difference in the treatment effect across nodes $\rightarrow$ suggests the following loss function:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (\tau_i - \widehat{\tau}(X_i))^2$$

This is unfortunately unobserved! Let's find a trick to replace τ_i (causal effect for individual i).

The loss function

We want to minimize the loss $\mathcal{L}$

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (\tau_i - \widehat{\tau}(X_i))^2$$

The loss function

We want to minimize the loss $\mathcal{L}$

$$\begin{aligned}\mathcal{L} &= \frac{1}{N}\sum_{i=1}^{N}(\tau_i - \widehat{\tau}(X_i))^2 \\ &= \frac{1}{N}\sum_{i=1}^{N}\tau_i^2 - 2\tau_i\widehat{\tau}(X_i) + \widehat{\tau}(X_i)^2\end{aligned}$$

The loss function

We want to minimize the loss $\mathcal{L}$

$$\begin{aligned}\mathcal{L} &= \frac{1}{N}\sum_{i=1}^{N}(\tau_i - \widehat{\tau}(X_i))^2 \\ &= \frac{1}{N}\sum_{i=1}^{N}\tau_i^2 - 2\tau_i\widehat{\tau}(X_i) + \widehat{\tau}(X_i)^2\end{aligned}$$

Now, note that, whatever the estimator, $\frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is a constant so minimizing $\mathcal{L}$ or $\mathcal{L} - \frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is the same.

The loss function

We want to minimize the loss $\mathcal{L}$

$$\begin{aligned}\mathcal{L} &= \frac{1}{N}\sum_{i=1}^{N}(\tau_i - \widehat{\tau}(X_i))^2 \\ &= \frac{1}{N}\sum_{i=1}^{N}\tau_i^2 - 2\tau_i\widehat{\tau}(X_i) + \widehat{\tau}(X_i)^2\end{aligned}$$

Now, note that, whatever the estimator, $\frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is a constant so minimizing $\mathcal{L}$ or $\mathcal{L} - \frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is the same... but it simplifies a lot the computations.

The loss function

Now, note that, whatever the estimator, $\frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is a constant so minimizing $\mathcal{L}$ or $\mathcal{L} - \frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is the same... but it simplifies a lot the computations:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\tau_i^2 - 2\tau_i\widehat{\tau}\left(X_i\right) + \widehat{\tau}\left(X_i\right)^2 - \frac{1}{N}\sum_{i=1}^{N}\tau_i^2$$

The loss function

Now, note that, whatever the estimator, $\frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is a constant so minimizing $\mathcal{L}$ or $\mathcal{L} - \frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is the same... but it simplifies a lot the computations:

$$\begin{aligned}\mathcal{L} &= \frac{1}{N}\sum_{i=1}^{N}\tau_i^2 - 2\tau_i\widehat{\tau}(X_i) + \widehat{\tau}(X_i)^2 - \frac{1}{N}\sum_{i=1}^{N}\tau_i^2 \\ &= \frac{1}{N}\sum_{i=1}^{N} -2\tau_i\widehat{\tau}(X_i) + \widehat{\tau}(X_i)^2\end{aligned}$$

The loss function

Now, note that, whatever the estimator, $\frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is a constant so minimizing $\mathcal{L}$ or $\mathcal{L}-\frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is the same... but it simplifies a lot the computations:

$$\begin{aligned}\mathcal{L} &= \frac{1}{N}\sum_{i=1}^{N}\tau_i^2 - 2\tau_i\widehat{\tau}(X_i) + \widehat{\tau}(X_i)^2 - \frac{1}{N}\sum_{i=1}^{N}\tau_i^2 \\ &= \frac{1}{N}\sum_{i=1}^{N} -2\tau_i\widehat{\tau}(X_i) + \widehat{\tau}(X_i)^2 \\ &= \frac{1}{N}\sum_{i=1}^{N}\widehat{\tau}(X_i)\left(\widehat{\tau}(X_i) - 2\tau_i\right)\end{aligned}$$

What is the main difference with the usual loss function $\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\widehat{y}\left(\widehat{y}-2Y_i\right)$

The loss function

Now, note that, whatever the estimator, $\frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is a constant so minimizing $\mathcal{L}$ or $\mathcal{L} - \frac{1}{N}\sum_{i=1}^{N}\tau_i^2$ is the same... but it simplifies a lot the computations:

$$\begin{aligned}\mathcal{L} &= \frac{1}{N}\sum_{i=1}^{N}\tau_i^2 - 2\tau_i\widehat{\tau}(X_i) + \widehat{\tau}(X_i)^2 - \frac{1}{N}\sum_{i=1}^{N}\tau_i^2 \\ &= \frac{1}{N}\sum_{i=1}^{N} -2\tau_i\widehat{\tau}(X_i) + \widehat{\tau}(X_i)^2 \\ &= \frac{1}{N}\sum_{i=1}^{N}\widehat{\tau}(X_i)\left(\widehat{\tau}(X_i) - 2\tau_i\right)\end{aligned}$$

What is the main difference with the usual loss function $\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\widehat{y}\left(\widehat{y} - 2Y_i\right)$

In the present case: ground truth (τ_i) is unobserved.

The loss function

Trick: for units i in leaf l of the tree, we have

$$\mathbb{E}\left(\tau_i \mid i \in l\right) = \mathbb{E}\left(\widehat{\tau}\left(X_i\right) \mid i \in l\right)$$

We thus have an unbiased estimate for the ideal loss function:

$$\widehat{\mathcal{L}} = \frac{1}{N} \sum_{i=1}^{N} \widehat{\tau}\left(X_i\right)\left(\widehat{\tau}\left(X_i\right) - 2\widehat{\tau}\left(X_i\right)\right)$$

The loss function

Trick: for units i in leaf l of the tree, we have

$$\mathbb{E}\left(\tau_i \mid i \in l\right) = \mathbb{E}\left(\widehat{\tau}\left(X_i\right) \mid i \in l\right)$$

We thus have an unbiased estimate for the ideal loss function:

$$\begin{aligned}\widehat{\mathcal{L}} &= \frac{1}{N}\sum_{i=1}^{N} \widehat{\tau}\left(X_i\right)\left(\widehat{\tau}\left(X_i\right) - 2\widehat{\tau}\left(X_i\right)\right) \\ &= \frac{1}{N}\sum_{i=1}^{N} -\widehat{\tau}\left(X_i\right)^2\end{aligned}$$

Why is that an unbiased estimate for the loss function? Explanations

Example

In the following slides, I am drawing on the code and example by Athey and Wager (2019). Note that I simplify quite a lot the coding part.

Context:

- A randomized study in U.S. public high schools.
- Designed to test a **growth mindset intervention**:
  - The intervention is a brief, nudge-like program that encourages students to believe that intelligence is malleable.
- Here: not the true data. Simulated dataset mimicking the original one.
- Key aspects of the dataset:
  - Approximately 10,000 student observations.
  - 76 schools.
  - Contains both student-level variables (e.g., academic achievement, self-reported expectations) and school-level characteristics.
- The objective: Estimate the treatment effect of the growth mindset intervention and explore heterogeneity across students and schools.

Data and Key Variables

- `Y`: Outcome measure (student achievement)
- `W`: Treatment indicator (1 if received the growth mindset intervention, 0 otherwise)
- `X`: Matrix of features / covariates (student and school characteristics)
- `school.id`: School identifier (used for clustering)

Available features

Variable	Definition
S3	Student's self-reported expectations for success in the future, a proxy for prior achievement
C1	Categorical variable for student race/ethnicity
C2	Categorical variable for student identified gender
X2	Categorical variable for student first-generation status, i.e. first in family to go to college
X3	School-level categorical variable for urbanicity of the school, i.e. rural, suburban, etc.
X1	School-level means of students' fixed mindsets, reported prior to random assignment
X5	School achievement level, as measured by test scores and college preparation for the previous 4 cohorts of incoming students
X4	School racial/ethnic minority composition, i.e. percentage of the student body that is Black, Latino, or Native American
X5	School economic disadvantage, as measured by share of students who are from families whose income falls below the federal poverty line
Y	Post-treatment outcome, a continuous measure of achievement
W	Treatment, i.e., receipt of the mindset intervention

Growing the causal tree

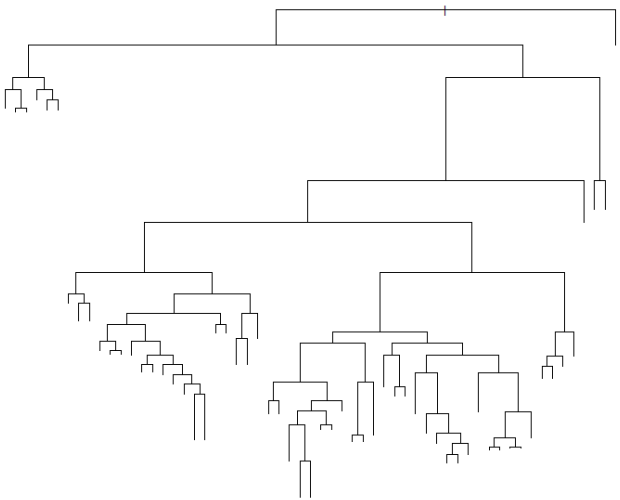
```r
#install.packages("htetree")
library(htetree)
library(rpart)

# prepare covariates vector
covariates <- colnames(df)
covariates <- covariates[- which(covariates %in% c("C1", "XC", "Y","Z"
    ,"schoolid"))]

# formula for the tree
fmla <- formula(paste("Y ~", paste(covariates, collapse=" + ")))

# Grow tree
causal_tree <- causalTree(formula = fmla,
             data=df,
             treatment = df$Z,
             split.Rule = "CT",
             cv.option = "CT",
             split.Honest = TRUE,
             cv.Honest = TRUE)
```

Plotting the tree

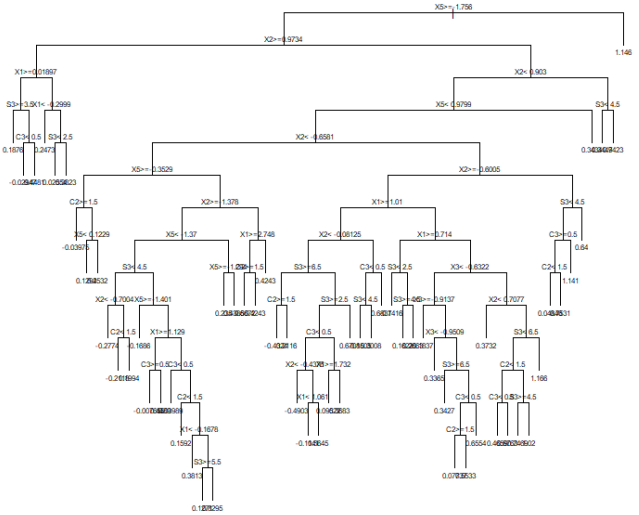


Growing the causal tree

```
## Find optimal complexity
cp.min <- which.min(causal_tree$ cptable[,"xerror"]) # minimum error
cp.idx <- which(causal_tree$ cptable[,"xerror"] - causal_tree$ cptable
    [cp.min,"xerror"] < causal_tree$ cptable[,"xstd"])[1]  # at most
    one std. error from minimum error
cp.best <- causal_tree$ cptable[cp.idx,"CP"]

# Prune the tree
pruned.tree <- prune(causal_tree, cp=cp.best)
```

Plotting the pruned tree



Questions ?

The honest approach

Don’t estimate the same way you split

In the simple tree algorithm, you use the following datasets: training dataset (split during cross-validation), test dataset.

Don't estimate the same way you split

In the simple tree algorithm, you use the following datasets: training dataset (split during cross-validation), test dataset.

But there is a **problem with the training set**: it's used to (i) find the splits, then (ii) estimate the outcome within leaf... $\rightarrow$ you decide where to split based on the outcome *in the specific training sample*, then you use this *same sample* to estimate the outcome values within the leaf. Is that really honest?

The honest approach

Use more datasets:

- Training dataset;
- Estimation dataset (to estimate outcome values within the leafs);
- Test dataset.

Note that the honest approach adds one sampling step to the estimation $\Rightarrow$ slightly changes the good loss function Explanations

Sad tree, good forest

Beyond a single tree

- Trees are very useful.

Beyond a single tree

- Trees are very useful...
- But one particular tree might have high-variance (be dependent on the sample). **Trees are known to have high-variance**.

$\Rightarrow$ Breiman (1996) proposed a method to aggregate predictors, resulting in a less noisy estimator. Wager and Athey (2018) extended it to causal trees, creating the concept of causal forests. More generally, random forests are an exemple of **ensemble methods**.

Why Ensemble Methods?

- No single model is perfect.
- Combining multiple models can reduce variance and bias.
- Main idea: **"Strength in numbers"** – aggregate several "weak learners" (= algo that performs only slightly better than random guess, see Künzel et al. (2019) for a reference) to form a "strong learner".
- 2 main approaches:
  - Boosting;
  - Bagging.

Boosting: Key Ideas

- Models are built sequentially.
- Each new model focuses on the mistakes made by the previous models.
- Combine models in a weighted manner to reduce bias.
- Popular algorithms: AdaBoost, Gradient Boosting.

Bagging: The Concept

- Create multiple datasets by sampling **with replacement** (bootstrapping) from the original data.
- Train a separate model (e.g., a regression tree) on each bootstrap sample.
- Aggregate the predictions:
  - **Regression:** Average the predictions.
  - **Classification:** Use majority voting.
- Reduces variance by averaging out errors.

$\Rightarrow$ Bagging means "bootstrap aggregating".

Bagging vs. Boosting

Bagging:

- Models are trained in parallel.
- Emphasizes reducing variance.
- Each model is independent.

Boosting:

- Models are trained sequentially.
- Focuses on reducing bias.
- Later models correct errors of earlier ones.

Random Forests: Introduction

- Random Forests are an extension of bagging applied to decision trees.
- In addition to bootstrapping data, they introduce randomness in feature selection.
- This randomness helps to decorrelate the trees, leading to improved performance.

Random Forest Algorithm

1. **For** $b = 1$ **to** B (number of trees):
   1.1 Draw a bootstrap sample from the training data.
   1.2 Grow a decision tree on this sample.
   1.3 At each node, randomly select a subset of features (typically, for classification, $m = \sqrt{p}$ and for regression, $m = p/3$, where p is the number of features).
   1.4 Split on the best feature among the selected subset.
2. **Aggregate:**
   - **Regression:** Average the predictions.
   - **Classification:** Use majority voting.
3. Causal forests:
   - Use causal trees;
   - Use the infinitesimal jackniffe estimator of variance;
   - Use subsampling (sampling witout replacement) instead of bootstrapping (sampling with replacement).

Mathematical Perspective

- Let the predictions of the B trees be $\hat{T}_1(x), \hat{T}_2(x), \ldots, \hat{T}_B(x)$.
- The Random Forest prediction is given by:

$$\hat{T}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{T}_b(x) \quad \text{(for regression)}$$

- For classification, the predicted class is the mode of $\{\hat{T}_1(x), \hat{T}_2(x), \ldots, \hat{T}_B(x)\}$.
- The averaging process reduces the variance if the individual trees are not too correlated.

Out-of-Bag (OOB) Error Estimation

- Since each tree is built on a bootstrap sample, about one-third of the data is left out (OOB data).
- OOB samples can be used as a validation set to estimate the model's error without the need for a separate test set.

Advantages and Limitations of Random Forests

- **Advantages:**
  - Robust to overfitting when enough trees are used.
  - Handles large datasets with high dimensionality.
  - Provides estimates of feature importance.
- **Limitations:**
  - Can be computationally intensive with a large number of trees.
  - The model can be less interpretable compared to a single decision tree.

Example in R

Back to our dataset. Let's use a random forest (not causal forest here, we don't have a treatment variable).

```
install.packages("grf")
library(grf)
set.seed(6545646) # for reproducibility
X <- df[,covariates]
Y <- df[,outcome]
random_forest <- regression_forest(X=X, Y=Y,
                                   num.trees=1000,
                                   honesty = TRUE,
                                   honesty.fraction = 0.5)
# if you want to tune parameters, you can do
random_forest <- regression_forest(X=X, Y=Y,
                                   num.trees=1000,
                                   honesty = TRUE,
                                   honesty.fraction = 0.5,
                                   tune.parameters="all")
```

Predicted values

Finding variables often chosen

```
var.imp <- variable_importance(random_forest)
names(var.imp) <- covariates
sort(var.imp, decreasing = TRUE)[1:10] # showing only first few
```

Observing individuals trees

```
1  install.packages("DiagrammeRsvg")
2  tree <- get_tree(random_forest, 1)
3  tree.plot = plot(tree)
4  cat(DiagrammeRsvg::export_svg(tree.plot), file = 'random_forest_
       tree.svg')
```

Graphing a tree

Why do you think we don't care about pruning the tree here?

Causal forest

Back to the growth mindset treatment.

```
library(grf)
X <- df[,covariates]
Y <- df[,Y]
W <- df[,Z]
causal_forest <- causal_forest(X=X, Y=Y, W=W,num.trees=1000,
                               honesty = TRUE,
                               honesty.fraction = 0.5,
                               tune.parameters="all")

# Estimate ATE
ATE <- average_treatment_effect(causal_forest)
paste("95% CI for the ATE:", round(ATE[1], 3),
    "+/-", round(qnorm(0.975) * ATE[2], 3))
```

Causal forest – Most chosen variables

```
var.imp <- variable_importance(causal_forest)
names(var.imp) <- covariates
sort(var.imp, decreasing = TRUE)
```

Variables the most split on: School-level means of students' fixed mindsets (prior to randomization), Categorical variable for student first-generation status (i.e. first in family to go to college), School economic disadvantage.

Causal forest – test for heterogeneity

```
library(grf)
X <- df[,covariates]
Y <- df[,outcome]
W <- df[,treatment]
causal_forest <- causal_forest(X=X, Y=Y, W=W,num.trees=1000,
                               honesty = TRUE,
                               honesty.fraction = 0.5,
                               tune.parameters="all")
test_calibration(causal_forest)
```

Techniques laid out in Chernozhukov et al. (2019) also allow to test for heterogeneity across subgroups, in particular best linear projections as discussed by Semenova and Chernozhukov (2021).

Conclusion

- For a long time: computer scientists uninterested in causal inference $\rightarrow$ ML not typically used by economists (except LASSO and ridge regressions);
- Over the past decade, growing interest and litterature about slight changes in algos that make **inference possible under reasonable assumptions**;
- Recent advances in the **estimation of CATEs**: causal trees, and causal forests.

To go beyond this class

- A rigorous and *technical* introduction to ML (only if you want to dig into the math): Hastie, Tibshirani, and Friedman (2017);
- A general beginner friendly introduction to more techniques and ML relevance: Wager (2024);
- The fundational paper about causal forests (generalized random forests): Athey, Tibshirani, and Wager (2019).

References

References

- Athey, Susan, Julie Tibshirani, and Stefan Wager. 2019. "Generalized Random Forests." *The Annals of Statistics* 47, no. 2 (April): 1148–1178. ISSN: 0090-5364, 2168-8966, accessed March 28, 2025. https://doi.org/10.1214/18-AOS1709.
- Athey, Susan, and Stefan Wager. 2019. *Estimating Treatment Effects with Causal Forests: An Application,* arXiv:1902.07409, February. Accessed March 28, 2025. https://doi.org/10.48550/arXiv.1902.07409. arXiv: 1902.07409 [stat].
- Breiman, Leo. 1996. "Bagging predictors" [in en]. *Machine Learning* 24, no. 2 (August): 123–140. ISSN: 0885-6125, 1573-0565, accessed March 15, 2025. https://doi.org/10.1007/BF00058655. http://link.springer.com/10.1007/BF00058655.

References ii

Chernozhukov, Victor, Mert Demirer, Esther Duflo, and Iván Fernández-Val. 2019. "Generic Machine Learning Inference on Heterogenous Treatment Effects in Randomized Experiments." Pre-published, (September 3, 2019). Accessed April 7, 2025. https://doi.org/10.48550/arXiv.1712.04802. arXiv: 1712.04802 [stat]. http://arxiv.org/abs/1712.04802.

Dell, Melissa. 2025. "Deep Learning for Economists" [in en]. *Journal of Economic Literature* 63 (1): 5–58. ISSN: 0022-0515, accessed March 15, 2025. https://doi.org/10.1257/jel.20241733. https://www.aeaweb.org/articles?id=10.1257/jel.20241733.

Hastie, Trevor, Robert Tibshirani, and Jerome H. Friedman. 2017. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction.* Second edition. Springer Series in Statistics. New York, NY: Springer. ISBN: 978-0-387-84857-0 978-0-387-84858-7.

References iii

- Künzel, Sören R., Jasjeet S. Sekhon, Peter J. Bickel, and Bin Yu. 2019. "Metalearners for Estimating Heterogeneous Treatment Effects Using Machine Learning." *Proceedings of the National Academy of Sciences* 116, no. 10 (March): 4156–4165. Accessed March 29, 2025. https://doi.org/10.1073/pnas.1804597116.

- Semenova, Vira, and Victor Chernozhukov. 2021. "Debiased Machine Learning of Conditional Average Treatment Effects and Other Causal Functions." *The Econometrics Journal* 24, no. 2 (May 1, 2021): 264–289. ISSN: 1368-4221, accessed April 7, 2025. https://doi.org/10.1093/ectj/utaa027. https://doi.org/10.1093/ectj/utaa027.

- Wager, Stefan. 2024. "Causal Inference: A Statistical Learning Approach."

References iv

Wager, Stefan, and Susan Athey. 2018. "Estimation and Inference of Heterogeneous Treatment Effects using Random Forests." Publisher: Taylor & Francis _eprint: https://doi.org/10.1080/01621459.2017.1319839, *Journal of the American Statistical Association* 113, no. 523 (July): 1228–1242. ISSN: 0162-1459, accessed June 1, 2022. https://doi.org/10.1080/01621459.2017.1319839. https://doi.org/10.1080/01621459.2017.1319839.

Annex

High-dimensionality

Back

Suppose you have p covariates, and n observations.

When $p > n$, impossible to perform regression $\Rightarrow X^\top X$ is singular. In this case, we say that the dataset is **high-dimensional**. Machine learning techniques can help in detecting the most relevant features and ignoring others.

Bias-variance decomposition

Back

Note that expectation with noise ϵ is both over samples S and noise. For simplicity, we use: $\mathbb{E} \equiv \mathbb{E}_{S,\epsilon}$

$$
\begin{aligned}
\mathbb{E}\left(\left(\widehat{Y} - Y\right)^2\right) &= \mathbb{E}\left(\left(X\widehat{\beta} - X\beta + \epsilon\right)^2\right) \\
&= \mathbb{E}\left(\left(X(\widehat{\beta} - \beta) + \epsilon\right)^2\right) \\
&= \mathbb{E}\left(\left(X(\widehat{\beta} - \beta)\right)^2 + \epsilon^2 + 2 \cdot \left(X(\widehat{\beta} - \beta)\right) \cdot \epsilon\right) \\
&= \mathbb{E}\left(\left(X(\widehat{\beta} - \beta)\right)^2\right) + \mathbb{E}\left(\epsilon^2\right) + 2\mathbb{E}\left(\left(X(\widehat{\beta} - \beta)\right) \cdot \epsilon\right) \\
&= \left(X(\beta - \mathbb{E}(\widehat{\beta})\right)^2 + \mathbb{E}\left(\widehat{Y} - \mathbb{E}\left(\left(\widehat{Y}\right)\right)^2\right) + \mathbb{E}(\epsilon^2)
\end{aligned}
$$

Bias-variance decomposition

$$\mathbb{E}\left(\left(\widehat{Y}-Y\right)^2\right)=\mathbb{E}\left(\left(X(\widehat{\beta}-\beta)\right)^2\right)+\mathbb{E}\left(\epsilon^2\right)+\underbrace{2\,\mathbb{E}\left(\left(X(\widehat{\beta}-\beta)\right)\cdot\epsilon\right)}_{=\mathbb{E}\left(X(\widehat{\beta}-\beta)\right)\underbrace{\mathbb{E}\left(\epsilon\right)}_{=0}}$$

$$=\mathbb{E}\left(\left(X(\widehat{\beta}-\beta)\right)^2\right)+\mathbb{E}\left(\epsilon^2\right)$$

We use independence of the noise (sometimes called irreducible error) and both parameters and estimate, and we also use that the mean of this error term is 0.

Bias-variance decomposition

$$\mathbb{E}\left(\left(\widehat{Y}-Y\right)^2\right)=\mathbb{E}\left(\left(X(\widehat{\beta}-\beta)\right)^2\right)+\mathbb{E}\left(\epsilon^2\right)$$

For simplicity (and generality), let's use: $f(X)=X\beta$.

$$\begin{aligned}\mathbb{E}\left(\left(X(\widehat{\beta}-\beta)\right)^2\right)=&\mathbb{E}\left(\left(f(X)-\mathbb{E}\left(\widehat{f}(X)\right)+\mathbb{E}\left(\widehat{f}(X)\right)-\widehat{f}(X)\right)^2\right)\\=&\mathbb{E}\left(\left(f(X)-\mathbb{E}\left(\widehat{f}(X)\right)\right)^2\right)+\mathbb{E}\left(\left(\mathbb{E}\left(\widehat{f}(X)\right)-\widehat{f}(X)\right)^2\right)\\&-2\mathbb{E}\left(\left(f(X)-\mathbb{E}\left(\widehat{f}(X)\right)\right)\left(\mathbb{E}\left(\widehat{f}(X)\right)-\widehat{f}(X)\right)\right)\end{aligned}$$

Bias-variance decomposition

$$\begin{aligned}\mathbb{E}\left(\left(X(\widehat{\beta}-\beta)\right)^2\right) =& \mathbb{E}\left(\left(f(X)-\mathbb{E}\left(\widehat{f}(X)\right)\right)^2\right)+\mathbb{E}\left(\left(\mathbb{E}\left(\widehat{f}(X)\right)-\widehat{f}(X)\right)^2\right)\\ &-2\mathbb{E}\left(\left(f(X)-\mathbb{E}\left(\widehat{f}(X)\right)\right)\left(\mathbb{E}\left(\widehat{f}(X)\right)-\widehat{f}(X)\right)\right)\end{aligned}$$

with:

$$\begin{aligned}\mathbb{E}\left(\left(f(X)-\mathbb{E}\left(\widehat{f}(X)\right)\right)^2\right) =& \mathbb{E}\left(f(X)^2+\mathbb{E}\left(\widehat{f}(X)\right)^2-2f(X)\mathbb{E}\left(\widehat{f}(X)\right)\right)\\ =& f(X)^2+\mathbb{E}\left(\widehat{f}(X)\right)^2-2f(X)\mathbb{E}\left(\widehat{f}(X)\right)\\ =& \left(f(X)-\mathbb{E}\left(\widehat{f}(X)\right)\right)^2\end{aligned}$$

Bias-variance decomposition

Back

$$\mathbb{E}\left(\left(X(\widehat{\beta}-\beta)\right)^2\right)=\left(f(X)-\mathbb{E}\left(\widehat{f}(X)\right)\right)^2+\mathbb{E}\left(\left(\mathbb{E}\left(\widehat{f}(X)\right)-\widehat{f}(X)\right)^2\right)$$
$$-2\mathbb{E}\left(\left(f(X)-\mathbb{E}\left(\widehat{f}(X)\right)\right)\left(\mathbb{E}\left(\widehat{f}(X)\right)-\widehat{f}(X)\right)\right)$$

with:

$$\begin{aligned}\mathbb{E}\left(\left(f(X)-\mathbb{E}\left(\widehat{f}(X)\right)\right)\left(\mathbb{E}\left(\widehat{f}(X)\right)-\widehat{f}(X)\right)\right)=&\mathbb{E}\left(f(X)\mathbb{E}\left(\widehat{f}(X)\right)\right)-\mathbb{E}\left(f(X)\widehat{f}(X)\right)\\&-\mathbb{E}\left(\mathbb{E}\left(\widehat{f}(X)\right)\mathbb{E}\left(\widehat{f}(X)\right)\right)-\mathbb{E}\left(\mathbb{E}\left(\widehat{f}(X)\right)\widehat{f}(X)\right)\\=&f(X)\mathbb{E}\left(\widehat{f}(X)\right)-f(X)\mathbb{E}\left(\widehat{f}(X)\right)\\&-\mathbb{E}\left(\widehat{f}(X)\right)^2-\mathbb{E}\left(\widehat{f}(X)\right)\mathbb{E}\left(\widehat{f}(X)\right)\\=&0\end{aligned}$$

Unbiasedness

Back

I omitted an important detail here: the estimator we plugged-in is computed using the validation sample V (not the training one), while the $\widehat{\tau}(X_i)$ initially in the formula are computed from the training sample Tr. These sample are independent from one another. Hence:

$$\mathbb{E}(\widehat{\tau}(X_i, Tr)(\widehat{\tau}(X_i, Tr) - 2\widehat{\tau}(X_i, V))) = \mathbb{E}(\widehat{\tau}(X_i, tr)^2 - 2\mathbb{E}(\widehat{\tau}(X_i, Tr) \cdot \widehat{\tau}(X_i, V))$$

Unbiasedness

Back

I omitted an important detail here: the estimator we plugged-in is computed using the validation sample V (not the training one), while the $\widehat{\tau}(X_i)$ initially in the formula are computed from the training sample Tr. These sample are independent from one another. Hence:

$$\begin{aligned}\mathbb{E}(\widehat{\tau}(X_i, Tr)(\widehat{\tau}(X_i, Tr) - 2\widehat{\tau}(X_i, V))) &= \mathbb{E}(\widehat{\tau}(X_i, tr)^2 - 2\mathbb{E}(\widehat{\tau}(X_i, Tr) \cdot \widehat{\tau}(X_i, V)) \\ &= \mathbb{E}(\widehat{\tau}(X_i, Tr)^2) - 2\underbrace{\mathbb{E}(\widehat{\tau}(X_i, Tr))\mathbb{E}(\widehat{\tau}(X_i, Tr))}_{\text{By independence}}\end{aligned}$$

Unbiasedness

Back

I omitted an important detail here: the estimator we plugged-in is computed using the validation sample V (not the training one), while the $\widehat{\tau}(X_i)$ initially in the formula are computed from the training sample Tr. These sample are independent from one another. Hence:

$$\begin{aligned}\mathbb{E}(\widehat{\tau}(X_i, Tr)(\widehat{\tau}(X_i, Tr) - 2\widehat{\tau}(X_i, V))) &= \mathbb{E}(\widehat{\tau}(X_i, tr)^2 - 2\mathbb{E}(\widehat{\tau}(X_i, Tr) \cdot \widehat{\tau}(X_i, V)) \\ &= \mathbb{E}(\widehat{\tau}(X_i, Tr)^2) - 2\underbrace{\mathbb{E}(\widehat{\tau}(X_i, Tr))\mathbb{E}(\widehat{\tau}(X_i, Tr))}_{\text{By independence}}\end{aligned}$$

Note that witin a leaf $\mathbb{E}(\tau_i \mid i \in I)$ is constant, and so is its estimator $\widehat{\tau}(X_i, Tr)$. Hence:

Unbiasedness

Back

I omitted an important detail here: the estimator we plugged-in is computed using the validation sample V (not the training one), while the $\widehat{\tau}(X_i)$ initially in the formula are computed from the training sample Tr. These sample are independent from one another. Hence:

$$\begin{aligned}\mathbb{E}(\widehat{\tau}(X_i, Tr)(\widehat{\tau}(X_i, Tr) - 2\widehat{\tau}(X_i, V))) &= \mathbb{E}(\widehat{\tau}(X_i, tr)^2 - 2\mathbb{E}(\widehat{\tau}(X_i, Tr) \cdot \widehat{\tau}(X_i, V)) \\ &= \mathbb{E}(\widehat{\tau}(X_i, Tr)^2) - 2\underbrace{\mathbb{E}(\widehat{\tau}(X_i, Tr))\mathbb{E}(\widehat{\tau}(X_i, Tr))}_{\text{By independence}}\end{aligned}$$

Note that witin a leaf $\mathbb{E}(\tau_i \mid i \in I)$ is constant, and so is its estimator $\widehat{\tau}(X_i, Tr)$. Hence:

$$\mathbb{E}(\widehat{\tau}(X_i, Tr))^2 = \mathbb{E}(\widehat{\tau}(X_i, Tr)^2)$$

Slight adjustments to the loss function

Back

Using the estimation dataset, you have **unbiased estimates** of outcomes within each leaf.

Slight adjustments to the loss function

Back

Using the estimation dataset, you have **unbiased estimates** of outcomes within each leaf... but these are *estimates*. $\Rightarrow$ Account for it in the loss criteria: penalty for leafs with few observations (treatment effects imprecisely estimated).

Denoting T the tree (partition of all leafs) with leafs $l \in T$, N^{tr} (resp. N^{est}) the number of units in the training (resp. estimation) dataset, the loss function to minimize is:

Slight adjustments to the loss function

Back

Using the estimation dataset, you have **unbiased estimates** of outcomes within each leaf... but these are *estimates*. $\Rightarrow$ Account for it in the loss criteria: penalty for leafs with few observations (treatment effects imprecisely estimated).

Denoting T the tree (partition of all leafs) with leafs $l \in T$, N^{tr} (resp. N^{est}) the number of units in the training (resp. estimation) dataset, the loss function to minimize is:

$$\mathcal{L} = -\frac{1}{N^{tr}} \sum_{i=1}^{N^{tr}} \widehat{\tau}_i^2 + \left(\frac{1}{N^{tr}} + \frac{1}{N^{est}} \right) \sum_{l \in T} \left(\frac{S^2_{D=1}(l)}{p} + \frac{S^2_{D=0}(l)}{1-p} \right)$$

where $S^2_{D=1}(l)$ (resp. $S^2_{D=0}(l)$) is the empirical variance of outcomes for treated (resp. control) units in leaf l.